

Project Report: Movie Review Sentiment Analysis (Classical ML)

Problem Statement

Understanding customer and audience opinion at scale is a critical need for businesses, streaming platforms, and content creators. Manually reading thousands of reviews to gauge sentiment is impractical, and misjudging overall sentiment can lead to poor content, product, or service decisions. This project aims to build a reliable, machine-learning powered tool to automatically classify the sentiment (positive or negative) of movie reviews, using classical natural language processing and machine learning techniques rather than deep learning.

Project Overview

We developed a machine learning model that analyzes the text of a movie review and predicts whether the sentiment expressed is positive or negative. The solution is deployed using a Streamlit web app, allowing users to interactively test the model with their own custom review text and instantly see the predicted sentiment along with a confidence score.

Dataset Summary

The dataset (IMDB Dataset.csv) contains 50,000 movie reviews sourced from IMDB, with the following structure:

- review — the full text of the movie review
- sentiment — the label, either positive or negative

Each record is labeled as:

- 1 — Positive
- 0 — Negative

The dataset was perfectly balanced (25,000 positive / 25,000 negative) with no missing values, though it contained 418 duplicate reviews which were removed during preprocessing. Standard text preprocessing steps — HTML tag removal, lowercasing, punctuation/number removal, negation-aware stopword removal, and stemming — were applied before feature extraction.

Exploratory Data Analysis (EDA Highlights)

- The dataset was perfectly class-balanced (50/50 positive vs. negative), so no class-imbalance handling was required.
- Average review length was around 230 words before cleaning, dropping to roughly 120 words after stopword removal and stemming.

- Raw reviews contained residual HTML formatting artifacts (e.g.,
 tags) that needed explicit cleaning.
- Negation words (e.g., "not", "no", "never") were found to carry strong sentiment-flipping signal and were deliberately preserved during stopwords removal, rather than being stripped out by default.

Model Development

We trained and compared multiple classical ML models including:

- Multinomial Naive Bayes
- Logistic Regression

Each was tested across three different feature representations:

- Bag-of-Words (unigrams)
- TF-IDF (unigrams)
- TF-IDF (unigrams + bigrams)

Logistic Regression with TF-IDF (unigrams + bigrams) yielded the best results with:

- Accuracy: ~89.9%
- F1 Score: ~0.90

This made it the final choice for deployment due to its strong, balanced performance and interpretability.

We used a pipeline to:

- Clean and normalize raw review text
- Vectorize text using TF-IDF with unigrams and bigrams
- Integrate the trained model and vectorizer seamlessly into the Streamlit app

Evaluation Metric

To evaluate model accuracy and robustness, we used:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

The confusion matrix showed a fairly balanced error distribution (563 false positives vs. 438 false negatives out of 9,917 test reviews), indicating the model was not systematically biased toward either sentiment class. Manual review of misclassified examples revealed that most errors occurred on reviews containing mixed sentiment, sarcasm, or comparisons to other unrelated films — a known limitation of classical, non-contextual ML approaches.

Streamlit App

The frontend is a lightweight, intuitive Streamlit app that allows users to:

- Input any movie review as free text
- Get a real-time sentiment prediction (Positive/Negative)
- View the model's confidence score as a percentage and progress bar
- Expand a breakdown of the full probability split between both classes

Positive predictions are shown with a 😊 emoji and Negative predictions with a 😞 emoji for quick visual interpretation.

Challenges

- Balancing thorough stopword removal with preserving negation words, which are critical for sentiment accuracy but often included in default stopword lists.
- Deploying scikit-learn and other dependencies to Streamlit Cloud, where an unpinned/incompatible Python version initially caused dependency installation to hang while compiling packages from source; this was resolved by explicitly selecting a well-supported Python version in the deployment settings.
- Interpreting model decisions for non-technical readers — addressed by extracting and presenting the top positive/negative predictive words and phrases learned by the model.

Impact

This tool can:

- Be integrated into review platforms or media monitoring dashboards to automatically flag overall audience sentiment
- Help content creators and studios quickly gauge audience reception at scale without manual review reading
- Serve as a learning prototype and baseline for building more advanced, context-aware sentiment analysis systems

Tech Stack

- Python
- Scikit-learn
- NLTK
- Pandas, NumPy
- Streamlit
- Joblib (for model and vectorizer persistence/deployment)

Future Work

- Incorporate lemmatization and compare its impact against stemming
- Add support for neutral/mixed sentiment detection instead of strictly binary classification
- Implement model interpretability tools like SHAP/LIME for deeper, per-prediction explanations
- Explore deep learning / transformer-based models (e.g., BERT) as a benchmark comparison against this classical ML baseline